6회차 · CDC / Iceberg 프로젝트 스터디

바뀐 것만 고치고도 전체 계산과 같은 답으로

모델 계약 · 컬럼 리니지 · 증분 범위 · 소급 정정과 재시도

약속한 열 모양을 지키고 영향받은 키와 날짜만 골라 Gold를 갱신합니다

흐름을 먼저 알고 설계한다

8회 단계적 진행 로드맵

STEP 1 · 1회차

개념 & 환경 구성

[핵심 학습]

- OT · 왜 CDC+Lakehouse인가
- CDC/Iceberg 내부 구조와 장단점
- 실습 환경 구성 및 공통 예제 흐름 체험

[프로젝트 진행 방식]

공통 예제 (orders·payments·refunds)로 전체 구조 파악

STEP 2 · 2~3회차

수집·적재 흐름 체득

[핵심 학습]

- 2회차: PostgreSQL → Debezium → Kafka 변경 수집
- 3회차: Flink → Iceberg Bronze 적재·지연 측정

[프로젝트 진행 방식]

공통 예제로 c/u/d·체크포인트·커밋을 직접 다룬 뒤, 3회차에 내 개인 프로젝트 도메인·테이블 설계 확정

STEP 3 · 4~6회차

내 프로젝트 본격 구현

[핵심 학습]

- 4회차: Spark SQL Silver 중복 제거·MERGE
- 5회차: dbt Gold 마트·품질 테스트
- 6회차: 과거 데이터 수정·삭제 소급 정정

[프로젝트 진행 방식]

내가 설계한 도메인 테이블과 지표를 파이프라인에 실제 투입·구현

STEP 4 · 7~8회차

성능 튜닝 & 운영 점검

[핵심 학습]

- 7회차: Iceberg 작은 파일 (rewrite_data_files)·파티션·쿼리 튜닝
- 8회차: Airflow 연동·장애 복구 점검·최종 리뷰

[프로젝트 진행 방식]

Iceberg의 단점(Small Files)을 직접 해결하는 전후 성능 비교 벤치마크 완성 (발표 별도)

5회차 결과에서 오늘 질문으로

수집 · 적재 (1~3회차)

오늘 집중 구간: 계약 · 리니지 · 증분 Gold



5회차 기준 구현

dbt build 한 번에 모델 5개 전체 교체 + 테스트 15개 (PASS=20)

오늘의 격리 실험판

공통 테이블은 그대로 두고 lake.lesson06에서 s1~s6 증분 실험

공통 파이프라인의 계산 규칙은 그대로 쓰고, 갱신 범위만 전체에서 영향 키·날짜로 좁힙니다

먼저 눈으로 볼 장면: 환불 한 건이 바뀔 때

직전 Gold 상태 (s2 직후)

o1 (11-01): 결제 100,000 · 환불 20,000 · 순 80,000
o2 (11-02): 결제 0 · 환불 0 · 순 0
o3 (11-02): 결제 50,000 · 환불 0 · 순 50,000

새로 들어온 변경 (s3)

refunds 테이블의 r1 한 행만 수정
amount: 20,000 → 30,000 (payment_id = p1)
orders · payments는 변경 없음

환불 테이블에는 order_id가 없고 payment_id만 있다는 점을 떠올려 보세요

데모: 영향 키 o1과 날짜 11-01만 MERGE하기

변경 감지

orders: []
payments: []
refunds: ['r1']

영향 범위 (s3)

keys: ['o1']
dates: ['2026-11-01']
o2·o3는 제외

전체 계산과 대조

o1 순결제 80,000→70,000
gold_order · daily 모두
EXCEPT 0 / 0 일치

```
-- labs/session06/sql/03-order-merge.sql 핵심
MERGE INTO {gold_order} t USING (
  SELECT k.order_id, n.order_date, n.paid, n.refunded, n.net_paid,
         n.order_id IS NOT NULL present
  FROM {affected_keys} k LEFT JOIN {full_order} n ON k.order_id = n.order_id
) s ON t.order_id = s.order_id
WHEN MATCHED AND NOT s.present THEN DELETE
WHEN MATCHED THEN UPDATE SET t.refunded = s.refunded, t.net_paid = s.net_paid ...
```

실측(11/2, s3) · 환불 한 건 수정 → keys=['o1'], dates=['2026-11-01']만 갱신 후 전체 계산과 일치

방금 데모가 안전하려면 필요한 두 가지

① 열의 약속과 계보

상류 열 이름·타입이 안 깨지고
어느 열이 어느 마트로 가는지
미리 알고 있어야 함

② 행의 증분 범위와 복구

수정뿐 아니라 삭제·날짜 이동·
실패 후 재시도에서도
영향 행을 놓치지 않아야 함

오늘 수업 순서

앞부분: dbt 계약과 컬럼 리니지
뒷부분: 증분 설계·운영 기준과
s1~s6 결과 확인 · 도메인 적용

거버넌스로 '열의 파급'을 확인한 뒤, 증분 설계로 '행의 파급'과 복구를 해결합니다

왜 데이터 테스트만으로는 부족할까

지키는 대상	어디에 적나	확인 시점과 실무 의미
열 이름·타입 (모델 계약)	_층_models.yml contract: enforced	테이블 만들기 전 · 합의 없는 스키마 변경 차단
데이터 내용 (키·관계·금액)	YAML tests · tests/*.sql (15개)	테이블 만든 뒤 · 하류 모델은 SKIP
설명·열 리니지·계약 현황	OpenMetadata 2.0.5 (선택 스택)	컴파일 SQL·검증 결과를 한 화면에

실측(11/1) · Silver core 3개 + Gold 2개 모두 contract: {enforced: true} 적용 후 PASS=20

모델 계약 위반 예측: amount 타입을 바꾸면?

원래 약속

fct_payment.amount
data_type: bigint
원천 SQL 결과도 BIGINT

→

위반 시연 조건

임시 복사본 YAML에서만
data_type: string으로 변경
dbt run --select fct_payment

→

예측 포인트

언제 실패할까?
기존 silver.fct_payment
테이블은 바뀔까?

```
# dbt/models/silver/_silver__models.yml (복사본 수정)
models:
  - name: fct_payment
    config:
      contract: {enforced: true}
    columns:
      - name: amount
        data_type: string   # 원래 bigint → string으로 변경해 실행
```

governance/contract-failure-demo.sh로 원본 프로젝트는 건드리지 않고 복사본에서 실험합니다

계약 위반 시연 결과와 Spark의 한계

사전 차단 확인

SQL은 BIGINT_TYPE
계약은 STRING_TYPE
테이블 변경 없이 즉시 실패

Spark 제약의 한계

dbt-spark 1.9.3에서
not_null 등 constraints는
모두 NOT_ENFORCED

우리 저장소의 분담

열 이름·타입 → contract
키 유일성·null·합계 →
data tests 15개

```
-- 1) contract-failure-demo.sh 실행 결과 (요지)
This model has an enforced contract that failed.
amount | BIGINT_TYPE | STRING_TYPE | data type mismatch

-- 2) 정상 계약 빌드 시 컴파일된 DDL (silver.dim_order)
create or replace table silver.dim_order using iceberg as
select order_id, ordered_at, order_date, status from (...)
```

실측(11/1) + 코드 확인 · Spark Iceberg DDL에는 제약이 붙지 않으므로 데이터 테스트가 계속 필요합니다

dbt의 약속을 OpenMetadata에 모으기

변환의 기준은 여전히 Git의 dbt 코드이고 OpenMetadata(2.0.5)는 그것을 읽어 보여 주는 카탈로그입니다

① dbt build

모델 5 · 테스트 15 실행
manifest · run_results 생성

② 테이블 수집

Spark Thrift 메타데이터
처리 18건 성공 (11/1)

③ dbt 수집

설명 · 테스트 결과 · 열 단위 리니지 24건 (11/1)

④ 계약 검증

5개 모델 데이터 계약 모두 Success (schema·설명)

실측(11/1) · catalog.json은 Iceberg 열 정보를 비워 두어 Spark 메타데이터와 YAML 타입을 함께 씁니다

컬럼 리니지: 열 하나가 어디까지 흘러가나

출발 열	중간 모델·열	도착 Gold 마트·열 (11/1 실측)
silver.orders.ordered_at	dim_order.order_date	주문 마트 · 일별 마트 order_date
silver.fct_payment.status	fct_refund.payment_status	주문 마트 환불 집계 조건
silver.fct_payment.amount	주문 마트 paid · net_paid	일별 마트 paid · net_paid
silver.fct_refund.amount	주문 마트 refunded · net_paid	일별 마트 refunded · net_paid

컴파일된 SQL을 해석해 열 단위 계보를 그립니다 · count(*)인 order_count만 원본 열이 없습니다

리니지에서 증분으로: amount 하나를 정정하면?

컬럼 리니지가 알려 주는 것

fct_payment.amount나 fct_refund.amount가
바뀌면 주문 마트와 일별 마트의
금액 열들이 영향을 받음 (열 단위)

본편에서 풀어야 할 질문

특정 결제·환불 한 건의 amount를 정정했을 때
두 Gold 테이블 전체를 다 다시 쓸까?
어디까지만 다시 계산하면 될까? (행 단위)

열의 계보에 키의 연결 경로를 더해야 '다시 계산할 행의 범위'가 나옵니다

① 언제 전체 재계산을 증분으로 바꿀까

전체 교체 (5회차)

매번 전체 읽기 · 전체 쓰기
로직이 단순하고 복구가 쉬움
감당되는 동안은 기본 선택

도입 전 판단 질문 (설계)

전체 재계산 시간 vs 목표 주기(30분 가정)
한 주기에 바뀌는 키·날짜 비율은?
영향 범위 계산 비용이 더 작은가?

전환과 대조 원칙

전체 재계산으로 정답 기준 먼저 확정
증분 결과를 기준과 대조
측정 전 속도 향상 단정 금지

변경 비율과 목표 주기(30분 가정), 범위 계산 비용을 먼저 따져 보고 전체 계산 기준 위에서 전환합니다

증분 버전이 통과해야 할 다섯 가지 조건

완료 조건 (6회차 기준)	놓치면 생기는 일	오늘 실험 단계
① 성공 전 진행 위치 미확정	실패 구간의 변경을 건너뜀	s6 실패·재시도
② 변경 키의 결제·환불 전체 재계산	차액만 더하다 중복·상태 꼬임	s1 · s2 · s3
③ 삭제된 키도 영향 범위에 포함	철회·삭제된 데이터가 Gold에 잔류	s4 환불 철회
④ 날짜 변경 시 옛·새 날짜 모두 갱신	옛 날짜에 유령 합계가 남음	s5 · s6 소급 정정
⑤ 실패 후 재시도해도 기준과 일치	주문 마트와 일별 마트가 어긋남	s6 정상 vs naive

오늘의 verify(EXCEPT 0/0)는 운영의 정기 전체 재계산 대조(reconciliation)를 축소한 장치입니다

② 변경을 어느 테이블에서 읽을까

후보 A: Silver core (dbt table)

dim_order · fct_payment · fct_refund
매번 CREATE OR REPLACE TABLE
이미 where not is_deleted 적용됨

후보 B: Silver 현재 상태 (MERGE)

orders · payments · refunds
바뀐 키만 LSN 가드로 MERGE
is_deleted = true 표시 보존

전체 교체 테이블의 스냅샷 차이는 업무 변경이 아닙니다 · Silver 현재 상태에서 읽습니다

Iceberg create_changelog_view로 변경 읽기

스냅샷 구간

start-snapshot-id 제외
end-snapshot-id 포함
첫 실행은 end만 지정

COW carry-over

파일이 다시 써질 때
그대로 복사된 이웃 행은
기본 동작으로 제거

전후 행 묶기

identifier_columns 지정
compute_updates = true
UPDATE_BEFORE / AFTER

```
CALL lake.system.create_changelog_view(
  table => 'lesson06.s06_demo_s5_orders',
  options => map('end-snapshot-id', '5247711452044697834',
                 'start-snapshot-id', '2118278423232506279'),
  changelog_view => 's06_..._changes_orders',
  identifier_columns => array('order_id'),
  compute_updates => true)
```

4회차에서 막혔던 overwrite 스냅샷의 증분 읽기를 changelog 뷰로 해결합니다

Explorer로 COW carry-over 직접 보기

s5 파일 재작성 snapshot

s1(o1·o2) · s2(o3)는 append
s5(o1 수정)만 overwrite
added 1 · deleted 1

s5 현재 데이터 파일

1행 파일: s2에서 추가된 o3
2행 파일: s5에서 다시 쓴
o1(LSN 500) + o2(LSN 100)

원본 .changes → 뷰

원본 .changes는 4행(o1·o2)
o2는 값·LSN 100 그대로 복사
changelog 뷰는 o1 2행만

```
-- Explorer: 6회차 · s5 원본 changes · o2 carry-over (_change_ordinal = 2 구간)
DELETE | 2 | 5247711452044697834 | o1 | 2026-11-01 01:00:00 | 100
INSERT | 2 | 5247711452044697834 | o1 | 2026-10-28 01:00:00 | 500  -- 실제 정정
DELETE | 2 | 5247711452044697834 | o2 | 2026-11-02 01:00:00 | 100  -- carry-over
INSERT | 2 | 5247711452044697834 | o2 | 2026-11-02 01:00:00 | 100  -- carry-over
```

실측(11/2, s5) · 파일이 다시 써질 때 따라온 o2를 create_changelog_view가 걸러 줍니다

③ 영향 키 전파: 환불 → 결제 → 주문

orders 변경

order_id가 바로 있음
그대로 영향 키에 추가

→

payments 변경

order_id가 바로 있음
그대로 영향 키에 추가

→

refunds 변경

order_id가 없음
payment_id로 결제를 거쳐
order_id를 찾아 추가

```
CREATE OR REPLACE TEMP VIEW {affected_keys} AS
SELECT order_id FROM {changes_orders}
UNION SELECT order_id FROM {changes_payments}
UNION SELECT p.order_id
    FROM {changes_refunds} r
    JOIN {input_payments} p ON r.payment_id = p.payment_id
```

하위 테이블의 변경을 마트의 기준 키(order_id)로 모으는 것이 키 전파입니다

활동: 내 도메인 키 전파 경로 한 줄 쓰기

항목	채팅에 적을 내용과 예시
공통 예제	refunds(payment_id) → payments(order_id) → 주문 마트(order_id)
내 도메인 경로	하위 테이블(조인 키) → 중간 테이블 → Gold 마트(기준 키)
삭제·철회 확인	하위 행이 삭제될 때 어느 현재 상태 테이블에서 연결 키를 찾을지

3분 활동 · 내 프로젝트의 '하위 변경 → 마트 기준 키' 경로와 삭제 시 키 확인 위치를 한 줄로 적습니다

왜 차액만 더하지 않고 영향 주문을 다시 합산할까

차액(delta) 누적 방식

'환불이 +10,000원 늘었으니
순결제에서 10,000원만 빼자'
결제 취소·환불 철회·재시도가 겹치면
부호 분기와 중복 방지가 복잡해짐

영향 키 전체 재집계 (이 실습)

'영향 키가 o1이니 o1에 속한
현재 유효 결제·환불 전체를 다시 더하자'
5회차 계산 식(02-full.sql)을 그대로 재사용
같은 범위를 다시 돌려도 금액이 같음

변경 감지로 대상 키만 좁히고, 선택된 키의 금액 계산은 전체 재계산과 같은 식을 씁니다

과거 날짜가 바뀌면 어느 날짜를 다시 묶을까

소급 정정 상황 (s5)

주문 o1의 ordered_at 정정
2026-11-01 → 2026-10-28
금액은 그대로, 날짜만 이동

새 날짜만 갱신하면

10-28 행은 새로 생기지만
11-01 행에 옛 100,000원이
유령처럼 그대로 남음

영향 날짜 세 원천

기존 Gold 주문의 날짜
U 새로 계산한 주문의 날짜
U 변경 전(UPDATE_BEFORE·DELETE)

```
-- lab.py affected_dates 계산
SELECT g.order_date FROM {gold_order} g JOIN {affected_keys} k ON g.order_id = k.order_id
UNION SELECT n.order_date FROM {full_order} n JOIN {affected_keys} k ON n.order_id = k.order_id
UNION SELECT to_date(from_utc_timestamp(ordered_at, 'Asia/Seoul')) order_date
    FROM {changes_orders} WHERE _change_type IN ('UPDATE_BEFORE', 'DELETE')
```

기준 열(order_date)이 바뀌는 정정은 반드시 '이전 값'과 '새 값' 양쪽 집계를 모두 갱신합니다

④ Gold 주문 마트와 일별 마트 MERGE 읽기

```
-- [1] 03-order-merge.sql: 주문 마트 (삭제된 주문도 지우기 위해 affected_keys 기준 LEFT JOIN)
MERGE INTO {gold_order} t USING (
  SELECT k.order_id, n.order_date, n.paid, n.refunded, n.net_paid, n.order_id IS NOT NULL present
  FROM {affected_keys} k LEFT JOIN {full_order} n ON k.order_id = n.order_id
) s ON t.order_id = s.order_id
WHEN MATCHED AND NOT s.present THEN DELETE
WHEN MATCHED THEN UPDATE SET t.order_date=s.order_date, t.paid=s.paid, t.refunded=s.refunded, t.net_paid=s.net_paid
WHEN NOT MATCHED AND s.present THEN INSERT (order_id,order_date,paid,refunded,net_paid) VALUES (...);


-- [2] 04-daily-merge.sql: 일별 마트 (남은 주문이 0건이 된 옛 날짜는 DELETE)
MERGE INTO {gold_daily} t USING (
  SELECT d.order_date, count(o.order_id) order_count, sum(o.paid) paid, sum(o.refunded) refunded, sum(o.net_paid) net_paid
  FROM {affected_dates} d LEFT JOIN {gold_order} o ON d.order_date <=> o.order_date GROUP BY d.order_date
) s ON t.order_date <=> s.order_date
WHEN MATCHED AND s.order_count = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET ... WHEN NOT MATCHED AND s.order_count > 0 THEN INSERT (...)
```

둘 다 '영향 목록'을 왼쪽에 두고 LEFT JOIN해 사라진 키(NOT present)와 빈 날짜(order_count=0)를 지웁니다

진행 위치(progress)와 계획(plan) 저장 순서

Iceberg 커밋은 테이블마다 따로 일어나므로 저장 순서와 스냅샷 보존이 복구를 결정합니다

① 끝 스냅샷 고정

progress에서 start 조회
각 입력 main ref에서 end 조회
VERSION AS OF로 고정

② 계획(plan) 저장

range_id(구간 해시)로
영향 keys · dates 목록을
Gold 쓰기 전에 저장

③ Gold 두 번 MERGE

gold_order 먼저 MERGE
이어서 gold_daily MERGE
(멈추면 plan만 남음)

④ progress 저장

두 MERGE가 모두 성공한
맨 마지막에만 갱신
스냅샷 만료 기간 주의

plan은 쓰기 전에, progress는 완료 후에 · 만료된 스냅샷에서는 이어 읽을 수 없습니다

왜 dbt incremental 대신 SQL로 먼저 볼까

dbt-spark 1.9.3 기본 incremental

append · merge · insert_overwrite · microbatch 지원
기본 merge에 WHEN MATCHED DELETE 분기 없음
다중 입력 스냅샷·마트 간 계획 보존은 별도

오늘 SQL로 펴서 본 이유

삭제·빈 날짜 DELETE와 plan·progress를 투명하게 확인
도메인에 삭제·날짜 이동이 없다면 dbt 기본도 적합
전략별 세부 비교표는 학생 가이드 4절에 수록

필요한 복구·삭제 장치를 SQL로 먼저 확인한 뒤 내 도메인 조건에 맞는 도구를 고릅니다

⑤ 고정 입력 실험판과 s1~s5 시나리오

단계	입력 변경 (UTC 시각 → 서울 날짜 동일)	검증 포인트
s1 초기	o1(11-01)·p1(100,000)·r1(20,000), o2(11-02, 결제 없음)	o1·o2 한 파일(2행) 적재 · 미결제 0원
s2 추가	o3(11-02)·p3(50,000) 추가 (LSN 200)	append로 o3 파일 추가 · 11-02 갱신
s3 수정	r1 환불액 20,000 → 30,000 (LSN 300)	r1 → p1 → o1 키 전파 (오프닝 데모)
s4 철회	r1 is_deleted = true (LSN 400)	환불 철회 반영 · 순결제 70,000→100,000
s5 정정	o1 ordered_at: 11-01 → 10-28 (LSN 500)	COW 재작성(o2 제외) + 옛 11-01 삭제

모든 시각은 오전 1시·3시(UTC)라 서울 시간(오전 10시·12시)으로 바꿔도 날짜가 같습니다

조회 전 예측: s4(환불 철회)와 s5(주문일 정정)

예측 항목	직전 단계 상태	채팅에 적을 것
s4 직후 o1의 net_paid	s3 결과: 결제 100,000 · 환불 30,000 · 순 70,000	r10이 is_deleted=true가 된 뒤 순결제액
s5의 영향 날짜(dates)	o1 주문일: 2026-11-01 → 2026-10-28	다시 집계해야 할 날짜 목록
s5 직후 gold_daily 날짜들	s4 결과: 2026-11-01(1건), 2026-11-02(2건)	최종 일별 마트에 남는 날짜 행

예측한 숫자와 날짜 목록을 다음 장의 Explorer 결과 표와 바로 맞춰 봅니다

실측 확인: s1~s5 증분 결과와 전체 계산 대조

s4 환불 철회

refunds: ['r1'] 감지
11-01 환불 0 · 순 100,000
verify 0 / 0 통과

s5 과거 날짜 이동

keys: ['o1'] (o2 제외)
dates: ['10-28', '11-01']
11-01 행 삭제 완료

s1~s5 전체 판정

고정 기대값(expected)과
전체 계산(EXCEPT) 모두
다섯 단계 연속 일치

{"step": "carryover", "table": "orders", "raw_rows": 4, "changelog_rows": 2, "carryover_keys": ["o2"]}
{"step": "plan", "keys": ["o1"], "dates": ["2026-10-28", "2026-11-01"]}
{"step": "verify", "gold_order": {"actual_minus_full": 0, "full_minus_actual": 0, "match": true},
"gold_daily": {"actual_minus_full": 0, "full_minus_actual": 0, "match": true}}
{"step": "expected", "scenario": "s5", "match": true,
"actual": {"gold_daily": [["2026-10-28", 1, 100000, 0, 100000], ["2026-11-02", 2, 50000, 0, 50000]]}}

실측(11/2) · 수정(s3)·철회(s4)·날짜 이동(s5) 모두 바뀐 키·날짜만 고쳐 전체 계산과 0/0 일치

⑥ 실패·재시도 함정 (s6): 두 마트 사이에서 멈추면?

s6 입력: o1 주문일이 2026-10-28 → 2026-10-25로 이동 · 주문 마트 MERGE 직후 일부러 예외 발생

① 1차 시도

o1 날짜 10-28 → 10-25
gold_order가 먼저 10-25로 갱신·커밋됨

→

② 강제 실패

gold_daily 갱신 직전 중단
gold_daily에는 아직 10-28 행이 남아 있음

→

③ 같은 범위 재시도

progress는 안 올렸으니
같은 스냅샷 구간으로 다시 실행 (8회차 재시도)

→

④ 계획 미저장 시

이미 바뀐 gold_order에서 옛 날짜를 찾으려 하면
10-25밖에 안 보임

progress를 안 올리는 것만으로는 부족합니다 · 이미 반쯤 바뀐 Gold를 보고 계획을 다시 짜면 틀립니다

s6 실측 비교: naive 불일치 vs 계획 재사용 복구

naive 모드 (s6 재시도)

dates: ['2026-10-25']만
gold_daily에 10-28 잔류
actual_minus_full = 1

정상 모드 (s6 재시도)

plan reused = true
dates: ['10-25', '10-28']
10-28 삭제 · verify 0 / 0

이 실험의 방어선 두 겹

① UPDATE_BEFORE 날짜 포함
② Gold 쓰기 전 plan 저장 후
같은 구간 재시도 시 재사용

```
-- [naive 모드 재시도 실측: 불일치 발생]
{"step": "plan", "reused": false, "persisted": false, "dates": ["2026-10-25"], "naive_dates": true}
{"step": "verify", "gold_daily": {"actual_minus_full": 1, "full_minus_actual": 0, "match": false}, "match": false}

-- [정상 모드 재시도 실측: 일치]
{"step": "plan", "reused": true, "keys": ["o1"], "dates": ["2026-10-25", "2026-10-28"]}
{"step": "verify", "gold_order": {"match": true}, "gold_daily": {"match": true}, "match": true}
```

실측(11/2) · 이 고정 입력 시나리오에서 쓰기 전 계획을 재사용해 재시도 결과가 전체 재계산과 같아졌습니다

내 프로젝트에 증분·정정·운영 기준 적용하기

설계·운영 질문	내 도메인으로 적을 것
① 도입 판단과 계약·원천	전체 재계산 시간 vs 주기 / contract 걸 모델 / 변경 읽을 현재 상태 테이블
② 키 전파 그래프	19장에 적은 하위→마트 기준 키 조인 경로 (삭제·철회 포함)
③ 이전·새 값이 둘 다 필요한 열	날짜·상태·카테고리 이동 시 옛 집계를 지워야 하는 열
④ 쓰기·복구·스냅샷 보존	DELETE 포함 MERGE, 쓰기 전 plan·완료 후 progress, 스냅샷 보존 기간
⑤ 정기 전체 재계산 대조	기준 뷰(full)와 EXCEPT 0 / 0을 대조할 시나리오와 운영 점검 주기

수업 뒤 집에서 demo.sh·Explorer와 lab.sh로 직접 확인한 뒤 내 도메인 기준표를 채웁니다

오늘 남길 것과 7회차 예고

거버넌스 확인

contract: 빌드 전 열 모양 차단
Spark 제약은 NOT_ENFORCED
컬럼 리니지로 열 파급 추적

증분·운영 기준

현재 상태 changelog + 키 전파
옛·새 날짜 갱신 + plan 선저장
정기 EXCEPT 대조와 실패 감지

7·8회차 연결

반복 COW MERGE와 파일 재작성
progress 스냅샷 만료 주의
7회차 유지 관리 · 8회차 재시도

다음 시간: 반복 MERGE로 쌓인 작은 파일과 스냅샷 유지 관리를 측정하고 비교합니다

[예비] Time Travel 교차 검증으로 변경 키 대조하기

왜 교차 검증했나

changelog 프로시저만 믿지 않고
start·end 두 시점 스냅샷을
직접 조인해 변경 키 대조

2차 실측 결과 (11/2)

s2(append): 둘 다 ['o3']
s5·s6(COW 재작성): o2 제외하고
둘 다 ['o1']만 일치

운영에서의 쓰임 (설계)

대용량 매 주기 실행은 비쌈
엔진 업그레이드·회귀 테스트의
검증 안전장치로 활용

```
-- lab.py crosscheck: 두 시점 스냅샷을 직접 비교
WITH b AS (SELECT * FROM {target} VERSION AS OF {start}),
     a AS (SELECT * FROM {target} VERSION AS OF {end})
SELECT coalesce(b.order_id, a.order_id) changed_key
FROM b FULL OUTER JOIN a ON b.order_id = a.order_id
WHERE b.order_id IS NULL OR a.order_id IS NULL OR NOT (b.col <=> a.col ...)
```

실측(11/2) · 데모 6세트 모든 단계 crosscheck 불일치 0건으로 두 방법의 변경 키가 같았습니다

[예비] 실패 뒤 재시도 전에 새 변경이 들어오면?

s6에서 실측한 범위

실패 직후 새 커밋 없이 재시도
(start, end) 구간이 그대로 유지
저장된 plan을 재사용해 일치

재시도 전 새 커밋 유입 시

입력 최신 스냅샷이 end2로 이동
새 range_id가 만들어져
이전 plan은 재사용되지 않음

설계 방어선과 미실측 한계

changes_orders의
UPDATE_BEFORE로
옛 날짜를 잡도록 작성(설계)
단 연속 수정 등은 미실측

설계(미실측) · 범위가 바뀌어 plan을 못 써도 UPDATE_BEFORE가 옛 날짜를 잡도록 짰으나 별도 검증이 필요합니다